PokéTrader

Team 3

Team Member 1:	Joseph Ko	joseph.ko@sjsu.edu
Team Member 2:	Isaiah Mak	isaiah.mak@sjsu.edu
Team Member 3:	Thaneesha Singh	thaneesha.singh@sjsu.edu

Functional Requirements

PokéTrader offers many different services for users. Every 12 hours, users can redeem a ticket to open a free Pokémon pack of any available set. Users can also check their virtual binder to see cards that they've already collected. To search for specific cards, users can type into the search bar to look for cards with specific characteristics (name, types, rarities). Users can also push out requests for trades and those requests can be accepted by other users who are interested in trading for particular cards. During a trade, users can chat with each other to confirm the trade and then confirm the trade of cards, which then the PokéTrader application will swap the corresponding cards in each person's binder to trade.

User View

<u>User Main Page (and Login)</u>

- Users can create a new account.
- Users must log in using their username and password in order to access their account.
- Users can log out.
- After login, the system creates a secure session for the user.
    - The first user using the account who is successfully logged into the system will be the only user who has access to the account. Other users who attempt logging into the same account will be denied access.

<u>Pack-Opening Page</u>

- Users will receive one free ticket per day (i.e. one for every 12 hours). In order to redeem the free packs, users must navigate to the "Pack Opening Page" and use their tickets to redeem a certain pack.
- Users can choose a Pack to open.
- On the top left of the page, there will be a message letting the user know when the next ticket can be earned via a timer.

<u>Binder Page</u>

- Users can use the PokéTrader sorting system to display a subset of cards in the user's binder collection. This binder sorting system which will include the following:
    - A search bar
        - The user can type in the search bar to search **(case-insensitive)** these particular characteristics:
            - Name
                - e.g. "charizard" → looks for cards named "Charizard"
                    - "name = charizard" also works
            - Card-Type

            - e.g. “card-type = pokemon” → looks for only Pokémon cards
            - e.g. “card-type = item” → looks for only Item cards
            - e.g. “card-type = energy” → looks for only Energy cards
        - Pokémon-Type
            - e.g. “pokemon-type = grass” → looks for only Pokémon cards that are Grass-types
        - Expansion
            - e.g. “set = SVI” → looks for cards only in the Scarlet and Violet set
                - “set = svi” also works
    - An “Advanced Options” button
        - Pulls up a selection of buttons to filter the following:
            - Card-Type
                - Pokémon
                - Trainer
                - Energy
            - Pokémon-Type
                - Grass, Fire, Water, Lightning, Psychic, Fighting, Darkness, Metal, Dragon, and Colorless
            - Expansion
                - Represented as 2-3 characters
                - e.g. Paldean Fates → PAF
                - e.g. Diamond & Pearl → DP
            - Rarity
                - Common (Black Circle ○)
                - Uncommon (Black Diamond ◇)
                - Rare (Black Star ★)
                - Double Rare (Double Black Star ★★)
                - Ultra Rare (Silver/Clear Stars ★★)
                - Illustration Rare (Single Gold Star ★)
                - Special Illustration Rare (Double Gold Stars ★★)
                - Hyper Rare (Triple Gold Stars ★★★)
                - Promo (Black Star with “PROMO”)
- Users also have the option to flip through their binder collection by clicking either a “right arrow” button on the right side of the binder or a “left arrow” button on the left side of the binder.

<u>Trade Page</u>

- Initiate a new trade request

    - The first user will fill out a short form that details what is being offered as a trade (i.e. one or more cards) and can optionally put what cards they want to trade for (status of the trade becomes Request).
- Accept a trade request
    - The second user will either decline the trade, indicating that they will not trade the card (status of the trade becomes Declined), or accept and request one or more cards from the first user (status of the trade becomes Pending Response).
    - The first user will either decline the trade, indicating that they are unwilling to continue with the trade (status of the trade becomes Declined), or accept and select one of the options that the second user requested.
    - The second user can decline the trade, indicating that they are unwilling to continue with the trade (status of the trade becomes Declined). Otherwise, the second user can accept the trade, to which both users are now in agreement, meaning the transaction will occur (status of the trade becomes Accepted).

Explanation of Entity Sets & Relationships

Entity Sets (6)

Users

- PK: Identified by the **username**.
- Attributes: A login **passwordHash** string (after the user's password is hashed) and a binary value to determine if they are **loggedIn** to the system. The number of **tickets** for redeeming a free pack from any available *Packs* can be earned by waiting until a designated time (**nextTicketTime**).
- Relations:
    - *Performs*: A *User* has a many-to-many relationship with *Trades*.
    - *Owns*: A *User* has a many-to-many relationship with *Cards*.
    - *Involves*: A *User* has a ternary many-many-many relationship with *Cards* and *Trades* (this is distinct from *Performs* and *Owns* because it serves a different purpose)
    - *Writes*: A *User* has a one-to-many relationship with *TradeMessages*.

Sets

- PK: Identified by the **setCode**, numbered 1, 2, 3, and so on.
- Attributes: The **setName** string for human-friendly identification.
- Relations:
    - *PackFromSet:* A *Set* has a one-to-many relationship with *Packs*.
    - *CardFromSet:* A *Set* has a one-to-many relationship with *Cards*.
    - *Contains*: A *Set* has a ternary one-to-many-to-many relationship with *Packs* and *Cards* as the one.

Packs

- PK: Identified by the **packID**.
- FK: The **setCode** from *Set* to represent the *PackFromSet* relation.
- Attributes: The **packName** string and a boolean representing whether the *Pack* is currently **available** for opening.
- Relations:
    - *Contains*: A *Pack* has a ternary one-to-many-to-many relationship with *Sets* and *Cards* as a many.
    - *PackFromSet:* A *Pack* has a one-to-many relationship with *Sets*.

Cards

- PK: Identified by the **cardID**.
- FK: The **setCode** from *Set* to represent the *CardFromSet* relation.
- Attributes: The **cardName**, Pokemon/Trainer **cardType**, card **subType** (e.g. Water, Fire, Item, Stadium), card **rarity**, and an **imageLink** to the card art.
- Relations:
    - *Contains*: A *Card* has a ternary one-to-many-to-many relationship with *Sets* and *Packs* as a many.
    - *CardFromSet:* A *Card* has a one-to-many relationship with *Sets*.
    - *Owns*: A *Card* has a many-to-many relationship with *Users*.
    - *Involves:* A *Card* has a ternary many-many-many relationship with *Users* and *Trades*.

Trades

- PK: Identified by the **tradeID**.
- Attributes: The **status** of the trade as Accepted, Denied, Pending response, and Request state.
- Relations:
    - *Has*: A *Trade* supports a weak one-to-many relationship with *TradesMessages*.
    - *Involves*: A *Trade* has a ternary many-many-many relationship with *Users* and *Cards*.
    - *Performs*: A *Trade* has a many-to-many relationship with exactly 2 *Users*.

TradeMessages

- PK: Identified by the **messageNum**.
- FK: Supported by the **tradeID** of the *Trade*.
- Attributes: The **timestamp** of the messages are recorded, the contents of the **message**, and the message **sender**.
- Relations:
    - *Has*: *TradeMessages* is supported by *Trade* because a message cannot be sent regarding a *Trade* without the original *Trade* existing.
    - *Writes*: A *TradeMessage* has a one-to-many relationship with *Users*.

Relations (4)

Performs

The *Performs* relationship connects the *Users* and *Trades* through a many-to-many relationship. A *User* can *Perform* many *Trades*, and a *Trade* can be *Performed* by exactly 2 *Users* (hence, we use the many designation). It will involve the **username** of the *User* and **tradeID** of the *Trade* as foreign keys to make up the table.

Involves

The *Involves* ternary relationship connects the *Users*, *Trades*, and *Cards* through a many-many-many relationship. A *Trade Involves* many *Cards* and many *Users* (exactly 2 as specified in *Performs*). A *Card* is associated with many *Users* (across different *Trades*) and can be *Involved* in many *Trades*. A *User* is associated with many *Cards* (across different *Trades*) and can be *Involved* in many *Trades*. It will involve the **tradeID** of the *Trade*, the **cardName** of the *Card*, and the **username** of the *User* as foreign keys to make up the table. The **quantity** requested will also be specified in this relationship.

Owns

The *Owns* relationship connects the *Users* and *Cards* through a many-to-many relationship. A *User* can *Own* many *Cards*, and a *Card* can be *Owned* by many *Users*. It will involve **username** of the *User* and **cardName** of the *Card* as foreign keys to make up the table as well as the **quantity** owned and how many copies of a *Card* (**numberOffered**) the *User* wants to offer for trades.

Contains

The *Contains* ternary relationship connects the *Sets*, *Packs*, and *Cards* through a one-to-many-to-many relationship. A *Pack* can *Contain* many *Cards*, and a *Card* can be *Contained* in many *Packs*. Each *Pack* and *Card* must come from the same *Set*. It will involve **setCode** of the *Set*, **packID** of the *Pack*, and **cardID** of the *Card* as foreign keys to make up the table.

Relations without Table (4)

PackFromSet

The *PackFromSet* relationship connects the *Packs* that belong to a *Set* through a one-to-many relationship. A *Pack* can only belong to one *Set*, but a *Set* can have multiple unique *Packs*. This will be represented by a foreign key under the *Packs* table referencing the *Set* it belongs to.

CardFromSet

The *CardFromSet* relationship connects the *Cards* that belong to a *Set* through a one-to-many relationship. A *Card* can only belong to one *Set*, but a *Set* can have many unique *Cards*. This will be represented by a foreign key under the *Cards* table referencing the *Set* it belongs to.

Has

The *Has* relationship connects the weak entity set *TradeMessages* to the *Trade* they are associated with through a one-to-many relationship. *TradeMessages* only exist if the original, singular *Trade* it belongs to remains stored in history. A *Trade* can have numerous back and forth *TradeMessages*. This will be represented by a foreign key under the *TradeMessages* table referencing the *Trade* it comes from, specifically, the primary key **tradeID** from *Trade*.

Writes

The *Writes* relationship connects the *TradeMessages* to *Users* through a one-to-many relationship. A *TradeMessage* is written by exactly one *User*, but a *User* can write many *TradeMessages*. This will be represented by a foreign key under the *TradeMessages* table referencing the *User* it was written by.

Relational Schema

Entity Sets

- Users(<u>username</u>, passwordHashed, loggedIn, nextTicketTime, tickets)
- Sets(<u>setCode</u>, setName)
- Packs(<u>packID</u>, packName, available, *setCode*)
- Cards(<u>cardID</u>, cardName, rarity, cardType, subType, imageLink, *setCode*)
- Trades(<u>tradeID</u>, status)
- TradeMessages(<u>messageNum</u>, <u>tradeID</u>, message, timestamp, *sender*)

Relations

- Performs(<u>username</u>, <u>tradeID</u>)
- Involves(<u>cardID</u>, <u>tradeID</u>, <u>username</u>, quantity)
- Owns(<u>username</u>, <u>cardID</u>, quantity, numberOffered)
- Contains(<u>packID</u>, <u>cardID</u>, setCode)